

Practical Transport Layer Security

SSL/TLS



SSL vs TLS

- SSL
 - Secure Sockets Layer
- TLS
 - Transport Layer Security

SSL vs TLS

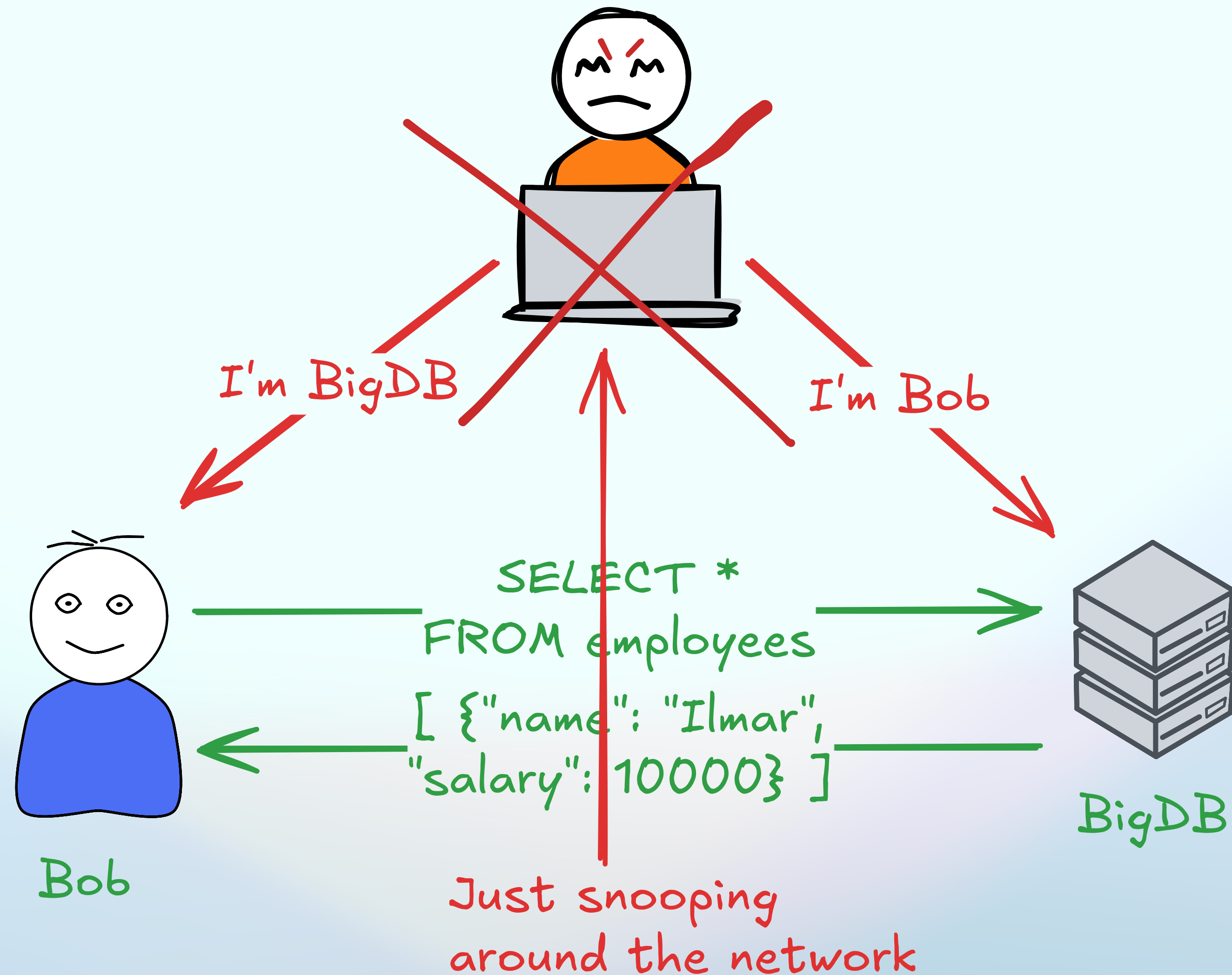
A little bit of history

- Netscape started to develop SSL in 1994
- SSL v3.0 (1996) was widely adopted
- IETF took over the standardisation of the protocol and published TLS 1.0 in 1999.
 - SSL is now deprecated.
- TLS 1.3 is the current standard. TLS 1.2 is also supported.
 - TLS 1.1 and 1.0 are deprecated.

The problem

- We want to enable end-to-end secure communication for our application
 - Securing data in transit
- Security and encryption are hard to implement
 - They must be implemented with utmost care
 - And audited

The problem



Why common TLS protocol?

- It is very hard to engineer security, encryption, authentication yourself
 - Implementation details matter
 - Research, auditing and attacks matter
 - Use a well known widely used library!
- TLS operates directly on TCP protocol
 - Anyone can add TLS to their network protocol to secure it

Securing data in transit

Adding security to any communication protocol



What TLS provides

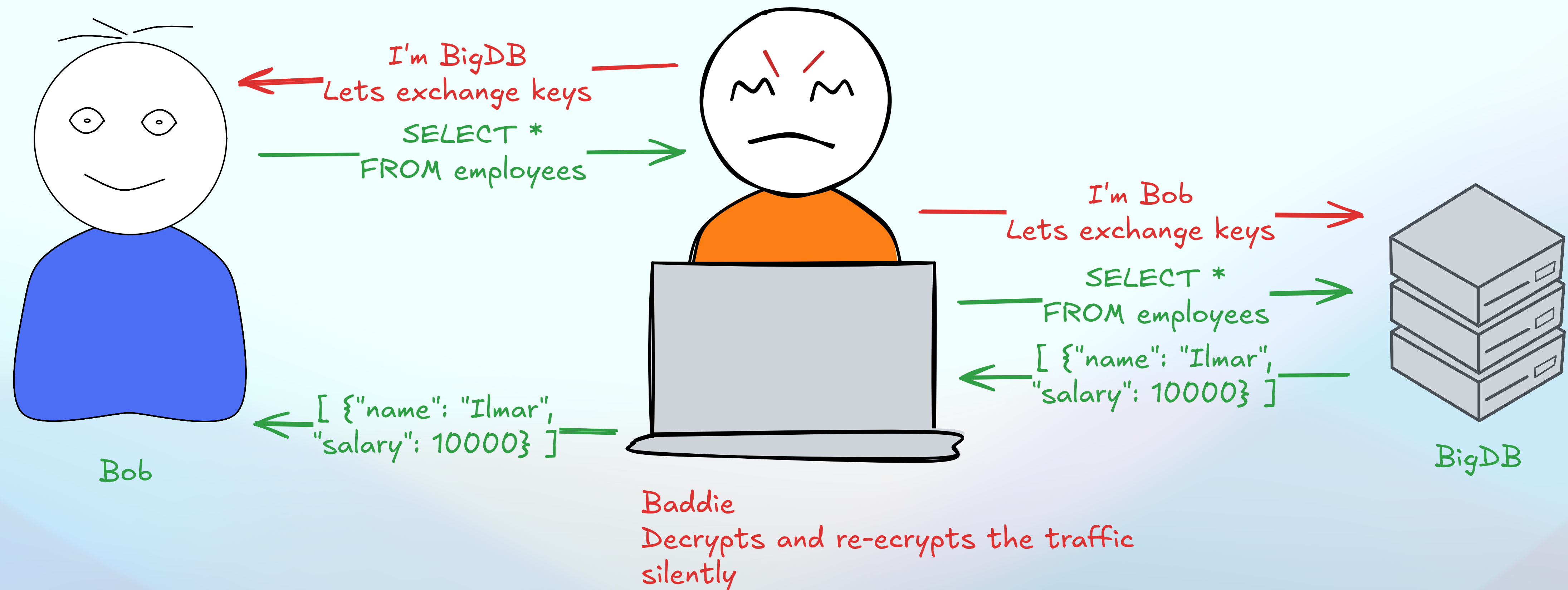
- Confidentiality
- Integrity
- Authenticity

Authenticity

- Need to make sure that every party is who they say they are
 - Avoid man-in-the-middle attacks
- Requires PKI and signed certificates
 - And client to check that certificate matches who they want to talk with
 - Server hostname client uses must be in certificate
- The feature that requires the most attention, creates confusion and work

Man-in-the-middle attack

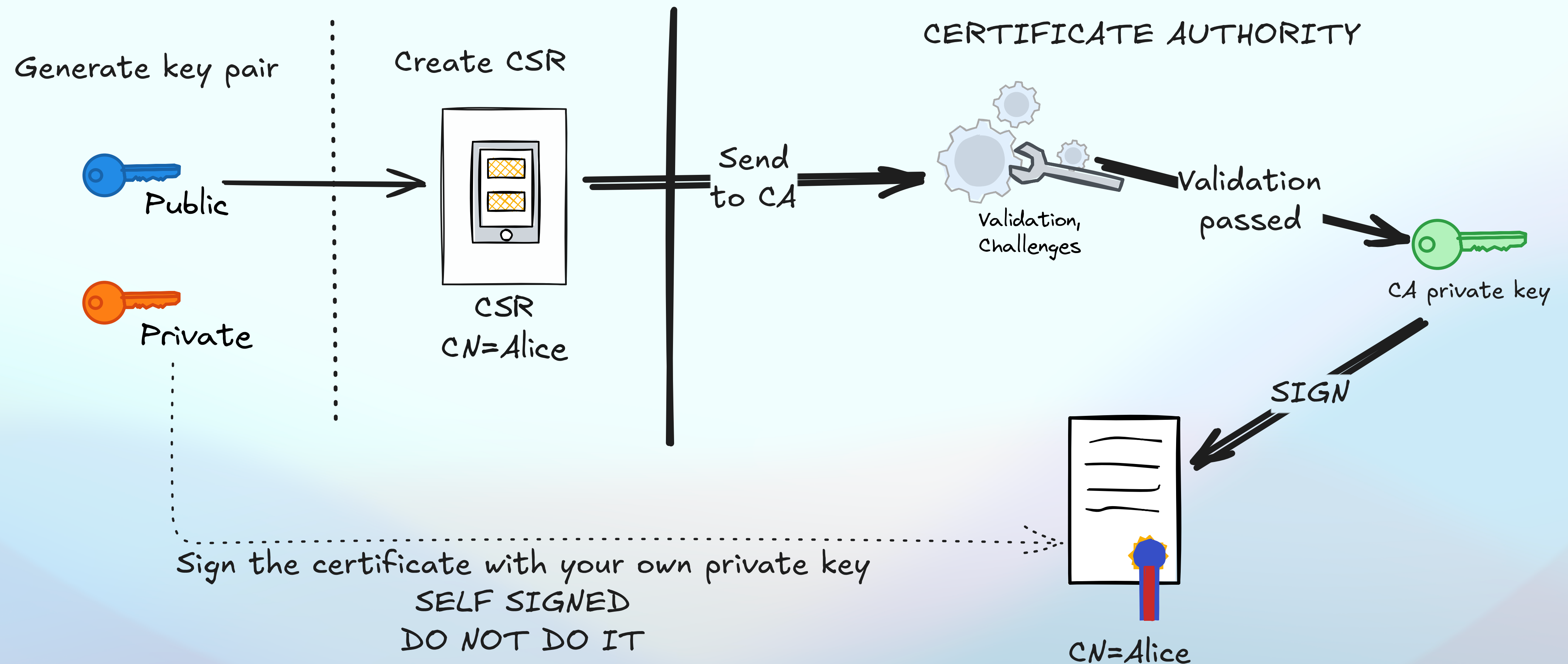
Non-repudiation



Certificates

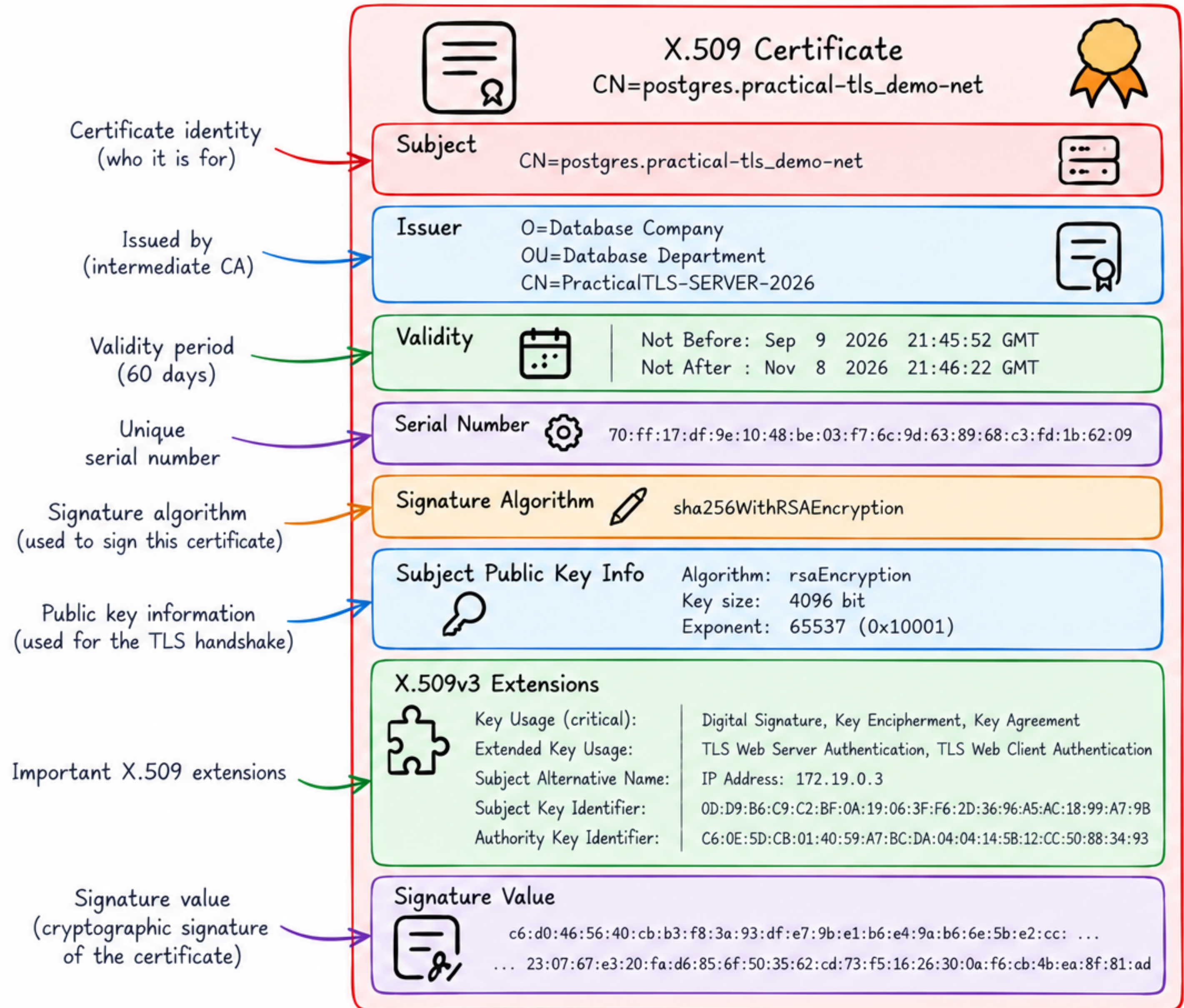
- What is certificate
 - Part of PKI, client needs to know the servers public key
 - Standard: X.509
- Scaling problem, how to distribute participant public keys in a trusted way?
 - Introducing Certificate Authorities
- Lazy option: skip CA and do a self-signed cert for each system
 - Don't be lazy

Creating a certificate process



Server Certificate

(CN=postgres.practical-tls_demo-net)



Oracle world

Server side

- What about Oracle Native Encryption - it is SOOO EEEAAASY compared to TLS!
 - Bulk encryption algorithm: AES-128/192/256 in CBC mode
 - Key exchange: DH
 - Data integrity algorithm: SHA-2
 - Anonymous, does not provide authenticity, vulnerable to man-in-the-middle
 - DPY-3001: Native Network Encryption and Data Integrity is only supported in python-oracledb thick mode
- Can be used in addition to TLS
 - IGNORE_ANO_ENCRYPTION_FOR_TCPS=TRUE

Oracle world

<https://github.com/oracle/python-oracledb/issues/477>



cjbj on Mar 28 Member ...

I appreciate you sharing the business case. We are still working on converting the request to allow us to support NNE so your request is useful for our argument. We have been discussing this a few days ago. The difference between JDBC and python-oracledb is that python-oracledb is a native driver.

NNE IS DEAD!

Overall, the Oracle DB security team would prefer users move to TLS. They have these resources:

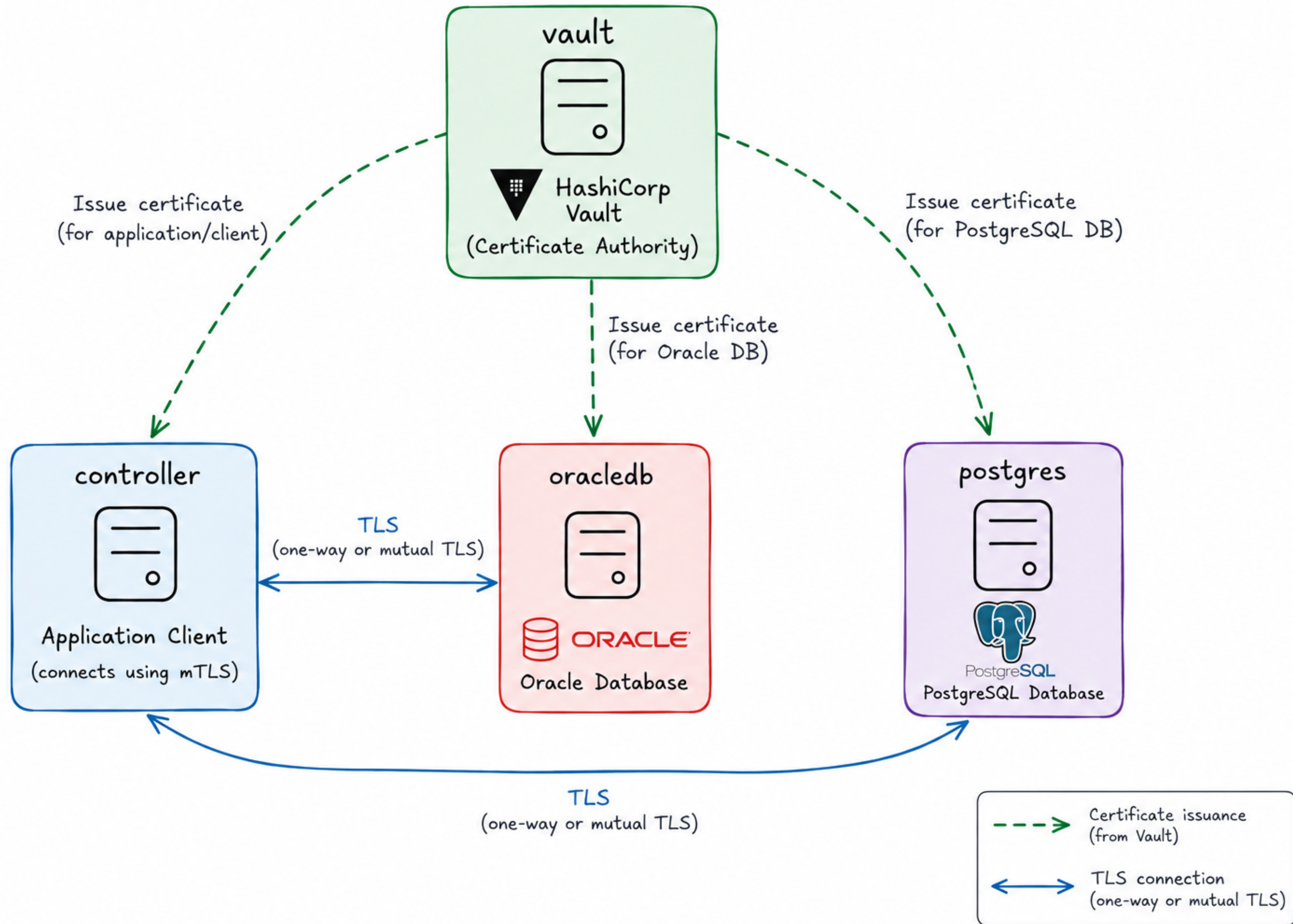
<https://docs.oracle.com/en/database/oracle/oracle-database/23/dbseg/configuring-transport-layer-security-encryption.html>

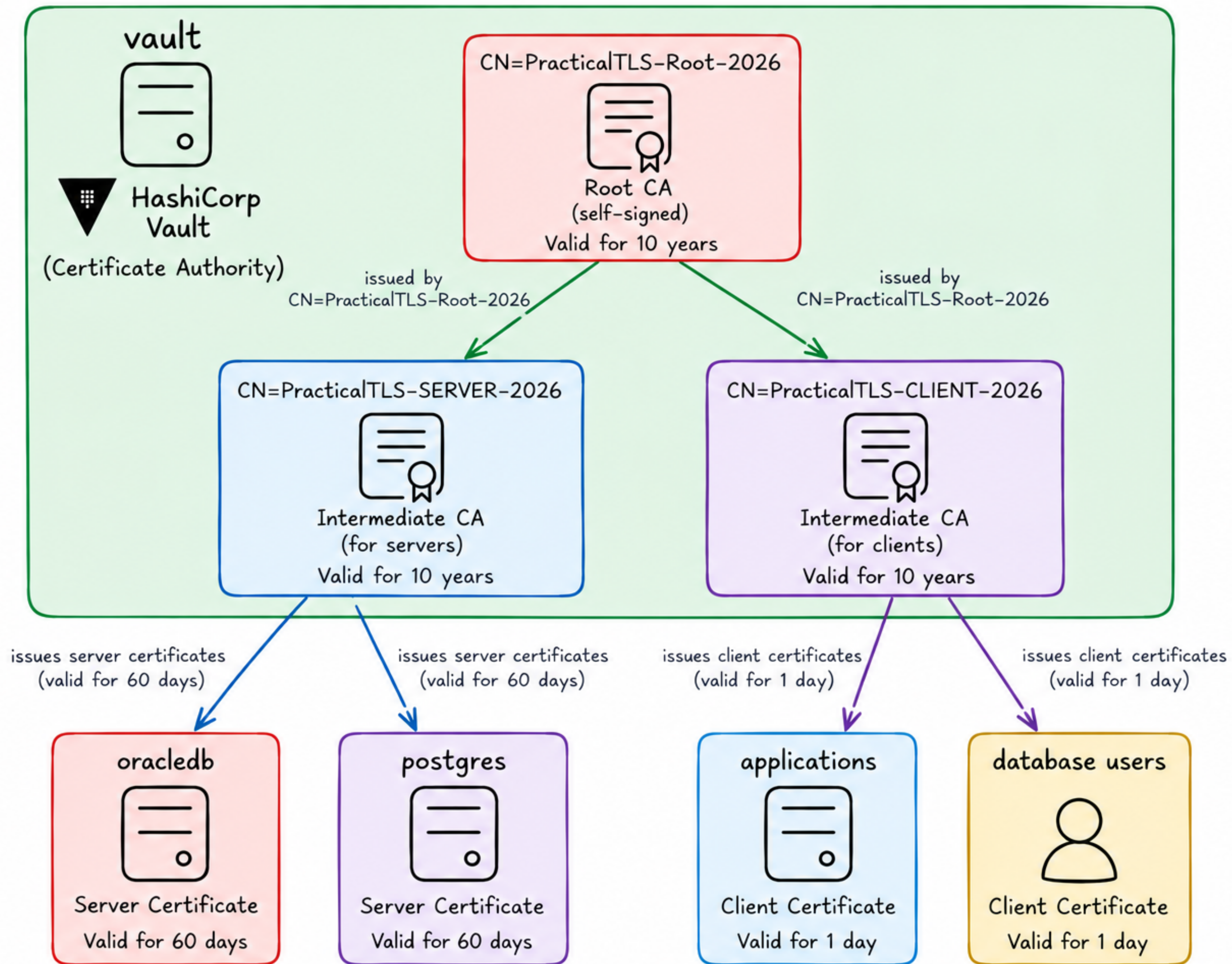
<https://livelabs.oracle.com/l/11236/0264/1/ARC/TLS.html>

<https://www.youtube.com/watch?v=1Gp2A...>

I'll close this issue but will continue to have NNE on the wish-list.

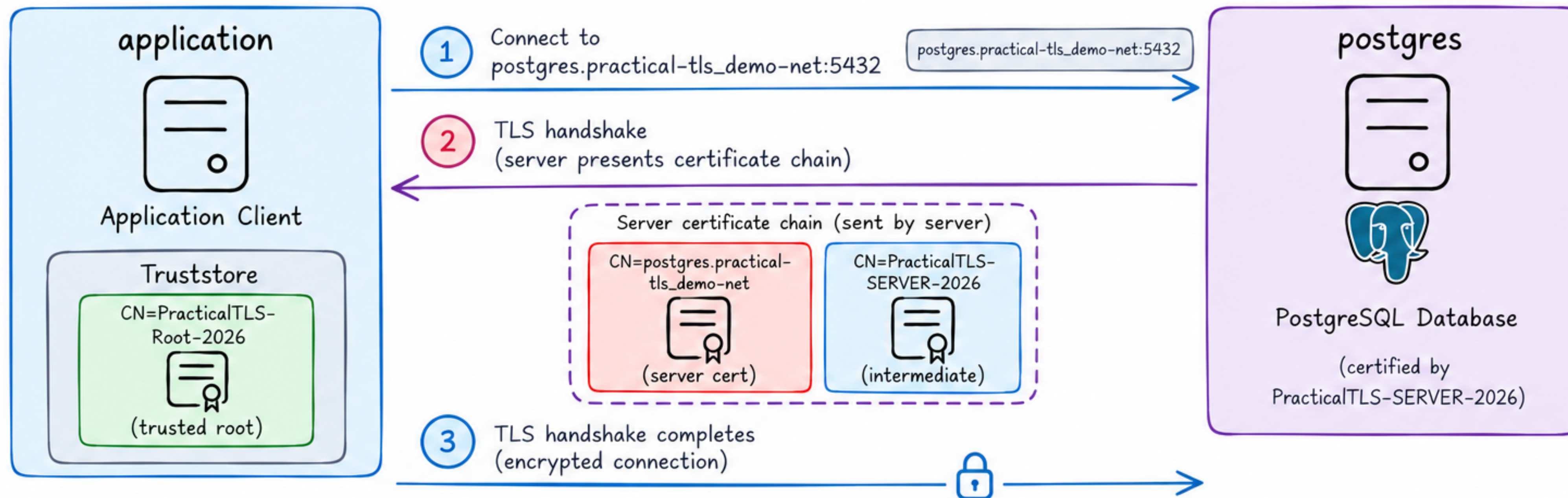
Also see [#94](#)



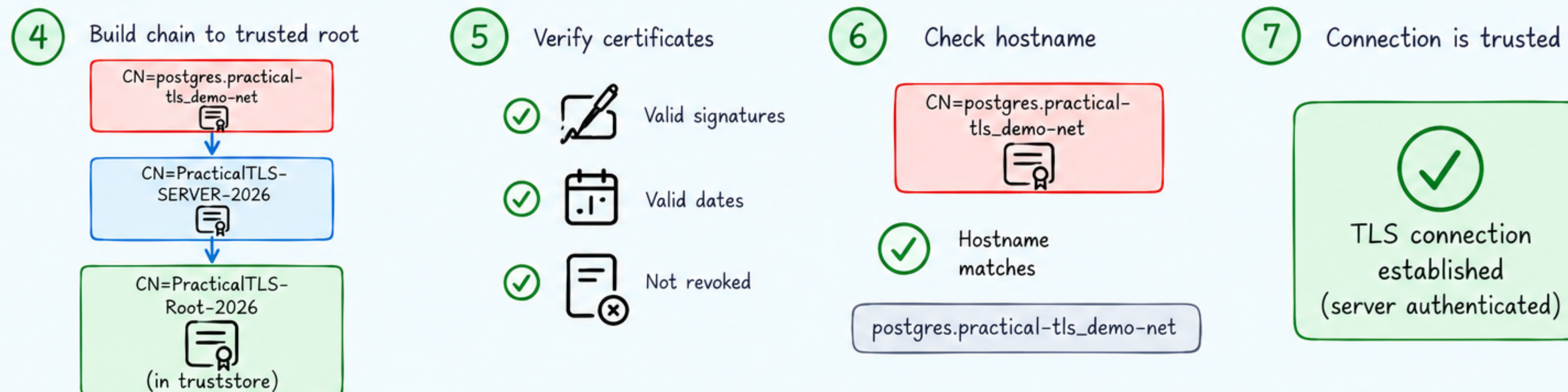


One-way TLS Connection

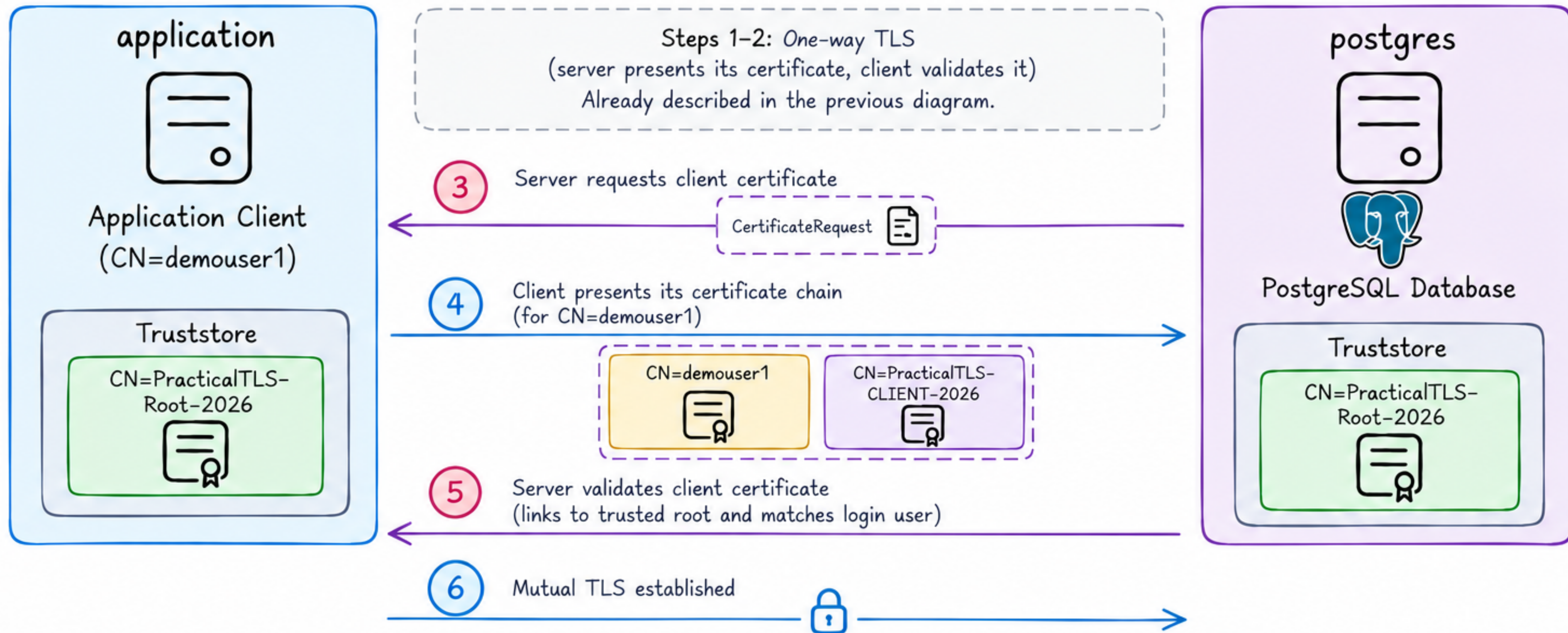
to postgres.practical-tls_demo-net:5432



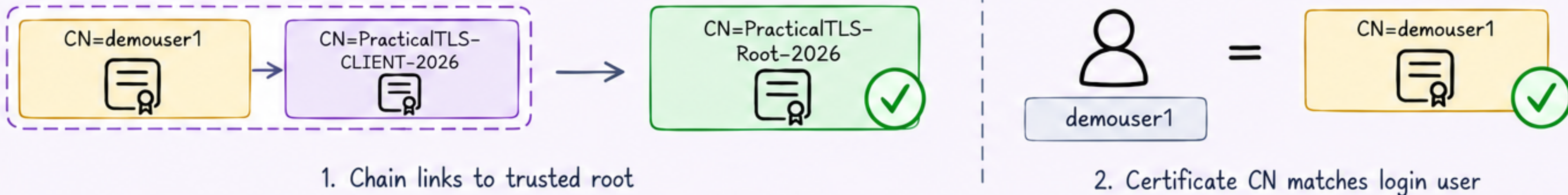
How the application validates the server certificate



Mutual TLS Connection (after successful one-way TLS)



Server validates the client certificate



**DEMO
TIME!**

